

CrimeGraph AI: Complete Project and Viva Study Guide

Purpose: a plain-language guide to the project, its code, datasets, models, metrics, demo, and honest answers for a viva. **Project type:** classroom full-stack and machine-learning prototype. **Current training data:** generated benchmark-style examples, not direct evaluation on the original named datasets.

First, remember this one-sentence summary

CrimeGraph AI is a web application that accepts sample notes, network-flow features, and hexadecimal file fragments; extracts or predicts structured information; stores case entities and relationships; and displays them as an interactive graph.

It is an investigation-support **prototype**. It does not establish guilt, verify evidence, or make decisions about people. Use synthetic examples only.

1. What the project contains

Frontend: what the user sees

The frontend is built with **React** and **Vite**. Its main screen contains:

- A case selector and a form for creating a case.
- API connection and analysis-engine readiness indicators.
- Summary cards for entity, relationship, person, and evidence-file counts.
- An interactive **Cytoscape.js** graph: nodes are records; edges are labeled relationships.
- Search, type filters, zoom, fit-to-view, and a detail panel for selected entities.
- An “Add evidence” workflow with three tabs:
- **Narrative:** submit notes for local pattern extraction (or optional Gemini if a server key is configured).
- **Network flow:** submit 15 traffic features for the CNN-LSTM classifier.
- **File bytes:** submit a file name and hexadecimal bytes for the TF-IDF + MLP classifier.
- JSON export of the active case and graph.

The browser does not load the Python models itself. It sends HTTP requests to the backend.

Backend: the API

The backend uses **FastAPI**, **Pydantic**, and **Uvicorn**.

- Pydantic checks inputs, including case IDs, numeric ranges, IP addresses, and hex data.
- FastAPI exposes the REST API and interactive documentation.
- The service layer performs text extraction, model inference, and graph updates.
- Model artifacts are loaded from the saved model files at startup.
- API errors are returned as HTTP errors instead of pretending an inference succeeded.

Important endpoints:

Method and route	Purpose
GET /api/v1/health	Backend and model readiness
GET /api/v1/cases	List cases
POST /api/v1/cases	Create a case
GET /api/v1/cases/{case_id}/graph	Return graph nodes and edges
POST /api/v1/cases/extract-narrative	Extract entities from note text
POST /api/v1/models/botnet/predict	Classify a network flow

Method and route	Purpose
POST /api/v1/models/file-classifier/predict	Classify a hexadecimal fragment

API documentation is at <http://127.0.0.1:8000/docs> when running locally.

Narrative extraction is not a trained ML model

The default narrative extractor uses local patterns/regular expressions to find supported examples of names, phone numbers, IP addresses, and location phrases. Gemini is optional and disabled unless a server-side API key is configured. If enabled, text is sent to Google's API.

The health endpoint counts this extractor as one of the three “analysis engines,” but only **two** engines are trained ML models: the traffic CNN-LSTM and file MLP.

Database and graph

The project uses **SQLite**, a small relational database file. It stores case metadata, graph nodes, and graph edges. Cytoscape renders the graph in the browser; SQLite is the storage, not the graph visualization.

The initial sample case is seeded on an empty database. It contains synthetic demo records (currently 10 nodes and 11 relationships). Names and labels in that sample are fictional demonstration content.

2. From data to a prediction: the two ML pipelines

A key dataset fact: what was actually collected?

The training scripts do **not** download or directly train on the original CTU-13, IoT-23, or Govdocs1 data in their normal run. They generate benchmark-style examples in code using selected patterns inspired by those datasets/file signatures. Do not say “I trained and validated on the full real CTU-13/IoT-23/Govdocs1 datasets.”

- Traffic generator: 10,000 synthetic flow rows. It simulates 60% benign and 40% malicious examples; malicious patterns include command-and-control beacon, DDoS, and port-scan styles. It uses 15 defined features.
- File generator: 6,000 synthetic hex fragments spread across six classes: PDF, DOCX, JPEG, executable, text log, and ZIP. Samples include class-associated byte-signature tokens plus noise.
- The file loader contains an optional local-directory path, but the training entry point calls it without a directory, so the normal run uses generated samples.
- Historical files that may exist in a local stash/notebook are not evidence that the active training script downloaded or evaluated the original corpora.

This is acceptable for demonstrating code workflow, but it limits what can be claimed from the scores. Synthetic test results are not measured forensic accuracy on real evidence.

Data split: train, validation, test

1. **Training set:** used to fit/adjust model parameters.
2. **Validation set:** checked during training; helps select the best checkpoint and adjust the learning-rate schedule.
3. **Test set:** held aside until the final evaluation; estimates performance on the benchmark examples not used for fitting.

The traffic script splits flows into approximately 70% train, 15% validation, and 15% test, then creates windows of 8 consecutive flows within each split. It fits the numeric feature scaler on training data only and applies that scaler to validation and test data.

The file script uses 64% training, 16% validation, and 20% test out of the 6,000 generated examples. It learns the TF-IDF vocabulary from training text only, then transforms validation/test with the same fitted vectorizer.

3. Model 1: network traffic classifier (CNN-LSTM)

Input and output

- **One example:** a sequence with shape (8 time steps, 15 features).
- **Batch input:** (batch size, 8, 15). With batch size 64, a typical batch is (64, 8, 15).
- **15 features:** duration; source/destination bytes; source/destination packets; byte and packet rates; TCP/UDP/ICMP indicators; common-port flag; inter-arrival time; SYN, FIN, and RST counts.

- **Output layer:** one raw number called a **logit**. Sigmoid turns it into a 0–1 malicious probability. The app labels it malicious at probability 0.5 or above; otherwise benign.

Architecture in order

8 × 15 input → Conv1D(64 filters, kernel 3) → BatchNorm + ReLU → bidirectional LSTM(hidden 64 each direction) → max over time → Dense(64) + ReLU + Dropout(0.35) → Dense(1 logit) → Sigmoid

- **Input layer:** receives the already prepared numeric sequence; it has no trainable weights by itself.
- **Convolution layer:** learns 64 filters that detect local patterns across neighboring sequence steps.
- **Batch normalization:** stabilizes intermediate activation scales during training.
- **ReLU:** non-linear activation, $\max(0, x)$.
- **Bidirectional LSTM:** processes sequence order in forward and reverse directions; 64 hidden values per direction, giving 128 combined values per time step.
- **Max pooling over time:** keeps the strongest activation for each of those 128 values.
- **Dense hidden layer:** maps 128 values to 64 learned activations.
- **Dropout:** randomly disables some activations during training to reduce overfitting; disabled at inference.
- **Output layer:** produces one logit for binary classification; sigmoid is applied for probability.

This hybrid has **77,953 trainable parameters** in its present configuration. These are learned weights and biases, not the number of training samples.

Training settings

- Framework: PyTorch.
- Loss: BCEWithLogitsLoss (binary cross entropy computed stably from logits).
- Optimizer: **Adam**, initial learning rate 0.001, weight decay 0.0001.
- Batch size: 64.
- Sequence length: 8.
- Epochs: 50 in the current run, meeting the lecturer's minimum.
- Scheduler: ReduceLROnPlateau; when validation loss stalls, it reduces learning rate by a factor of 0.5 after the configured patience.
- The best validation-F1 checkpoint is saved for inference.
- NumPy and PyTorch random seeds are set to 42 for repeatability, though exact results can still vary by software/hardware.

An **epoch** is one pass through the training data. It does not mean "50 samples"; every epoch processes all training batches once.

Traffic metrics from the current 50-epoch run

These are test results on the generated benchmark. CSVs are separate by model:

CSV	Model	Accuracy	Precision	Recall	F1	ROC-AUC
standalone_cnn_metrics.csv	Standalone CNN	98.40%	98.39%	100.00%	0.9919	0.9843
standalone_lstm_metrics.csv	Standalone LSTM	98.13%	98.64%	99.45%	0.9905	0.9836
hybrid_cnn_lstm_metrics.csv	Hybrid CNN-LSTM (deployed)	98.40%	98.65%	99.73%	0.9918	0.9860

The current run does **not** show the hybrid is best in every metric: the standalone CNN ties for highest accuracy, while the hybrid has the highest ROC-AUC. Do not repeat the older hard-coded baseline scores; the training script now computes results for each architecture on the generated test split.

4. Model 2: file-fragment classifier (TF-IDF + MLP)

Input and output

- **Input:** text containing hexadecimal bytes, for example 4d 5a 90 00
- TfidfVectorizer turns tokens and neighboring token pairs (1-grams and 2-grams) into up to 500 weighted numeric features.
- MLP input width in this run: **500**.

- MLP hidden layers: 128 units, then 64 units, using ReLU.
- Output width: **6 class scores** for PDF, DOCX, JPEG, executable, text log, and ZIP. The selected class is the highest predicted probability.
- Trainable MLP weights and biases: **72,774** for 500 inputs and six classes.

TF-IDF in plain English

Term frequency says how often a token appears in one sample. **Inverse document frequency** down-weights tokens common across many samples. The result is a vector that highlights informative byte tokens/pairs. The MLP learns relationships between those numbers and the file categories.

Training settings

- Framework: scikit-learn.
- MLP: hidden layers (128, 64), ReLU, Adam solver, learning rate 0.001, batch size 64.
- Training: `partial_fit` called exactly 50 times; each full call processes the training data using minibatches, so it is one training epoch.
- Loss: scikit-learn MLP classification log loss (cross entropy).
- Random Forest baseline: 100 trees, maximum depth 20. It is fitted once; epochs do not apply to tree ensembles.
- The app uses the saved TF-IDF vectorizer and MLP.
- The training script calculates SHAP feature attribution for the Random Forest baseline. The deployed app shows simple recognized-header diagnostics instead; do not call those displayed diagnostics "SHAP."

File metrics from the current 50-epoch run

CSV	Model	Epochs	Accuracy	Weighted precision	Weighted recall	Weighted F1	Macro F1
mlp_metrics.csv	TF-IDF + MLP	50	99.92%	99.92%	99.92%	0.9992	0.9992
random_forest_metrics.csv	Random Forest baseline	N/A	99.92%	99.92%	99.92%	0.9992	0.9992

The high score is plausible on generated data because each class has distinctive tokens that are present in both training and test examples. That is also why it must not be sold as real-world accuracy.

5. Metrics: what to say if asked

Let "positive" mean the class being measured (for traffic, malicious; for multiclass file predictions, compute each class then average).

- **Accuracy** = correct predictions / all predictions. Can be misleading when classes are imbalanced.
- **Precision** = true positives / all predicted positives. Of the examples flagged positive, how many were correct?
- **Recall** = true positives / all actual positives. Of the actual positives, how many did the model find?
- **F1 score** = harmonic mean of precision and recall: $2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$.
- **Macro F1**: calculate F1 per class, then average classes equally.
- **Weighted F1**: calculate per-class F1, then average weighted by how many examples each class has.
- **ROC-AUC**: measures how well positive examples tend to receive higher scores than negative ones across thresholds. 0.5 is random ranking; 1.0 is perfect ranking on that evaluation set.
- **Loss**: a training objective penalizing wrong/confident predictions; lower is generally better, but compare on the same task/split.
- **Confidence**: the model's score for a prediction, not a guarantee that it is correct.

The training CSV values are decimal fractions between 0 and 1 (e.g., accuracy 0.984 = 98.4%). Use the CSV from the latest run, not stale slide or README numbers.

6. Key ML and coding vocabulary

- **Pipeline**: ordered transformations and operations: raw input → validation/preprocessing → feature vector → model → prediction → API response/graph update.
- **Feature**: an input measurement/token the model uses.
- **Label**: the expected answer used during supervised training.
- **Input layer**: where prepared feature values enter a neural network.

- **Hidden layer:** intermediate learned representation between input and output.
- **Output layer:** produces the final score/logit/class scores.
- **Trainable parameter:** a weight or bias adjusted by the optimizer during training.
- **Forward pass:** compute the model output from inputs.
- **Loss:** numeric measure of how wrong predictions are relative to labels.
- **Backpropagation:** compute gradients indicating how parameters contributed to loss.
- **Optimizer:** applies gradient-based updates to parameters. The project uses Adam for both neural models; it is not used for Random Forest.
- **Learning rate:** step size for parameter updates. Too high may overshoot; too low may learn slowly.
- **Batch:** subset of examples processed for one gradient update.
- **Epoch:** one full pass over the training set.
- **Overfitting:** model performs well on training examples but poorly on new examples.
- **Validation set:** guides training choices without being the final score.
- **Test set:** final held-out evaluation.
- **Inference:** use a trained model to predict for a new input; no parameter updates happen.
- **Checkpoint:** saved model state, often the best validation-performing state.
- **Preprocessing/scaler:** converts inputs to the representation/range expected by training; inference must use the same fitted transformer.
- **API:** agreed request/response interface between frontend and backend.
- **REST endpoint:** URL + HTTP method for a backend operation.
- **CORS:** browser security rules that allow the published frontend origin to call the API.
- **Node/edge:** graph item/relationship.
- **SQLite:** file-backed relational database, separate from the on-screen Cytoscape graph.

7. Code workflow: where to look

Path	Responsibility
frontend/src/App.jsx	React state, API calls, case/evidence forms, graph rendering, search/filter/export
frontend/src/App.css, index.css	Visual styling and responsive layout
frontend/vite.config.js	Development server and /api proxy to localhost port 8000
backend/app/main.py	FastAPI setup, CORS, startup model/database initialization, route mounting
backend/app/api/endpoints.py	Health/cases/narrative/model REST endpoints
backend/app/schemas/graph.py	Pydantic request/graph types and validation
backend/app/services/llm_service.py	Optional Gemini and local narrative pattern extraction
backend/app/services/ml_service.py	Load saved model/scaler/vectorizer; prepare inference input; run predictions
backend/app/services/graph_service.py	Seed case, SQLite tables, deduplicate and persist graph changes
models/botnet_cnn_lstm/dataset.py	Generate traffic benchmark, split/scale features, create windows
models/botnet_cnn_lstm/model.py	CNN, LSTM, hybrid neural network architectures
models/botnet_cnn_lstm/train.py	Train/evaluate traffic architectures, save checkpoints, plots, JSON and CSV
models/file_classifier_mlp/dataset.py	Generate file-byte benchmark; optional local-file loader
models/file_classifier_mlp/model.py	TF-IDF, Random Forest, MLP definitions and artifact save/load
models/file_classifier_mlp/train.py	Split data, train baseline/MLP, evaluate, export CSV and SHAP plot
render.yaml	Render API configuration
.github/workflows/deploy-pages.yml	Build and deploy frontend to GitHub Pages

End-to-end traffic request

1. A user submits the traffic form in React.

2. React sends JSON to POST /api/v1/models/botnet/predict.
3. Pydantic validates the IPs and numeric features.
4. MLInferenceService calculates rates, applies the saved training scaler, and forms an 8-step input sequence.
5. The saved hybrid CNN-LSTM runs in inference mode and returns a threat probability/class.
6. GraphService adds/updates source and destination IP nodes and a relationship in SQLite.
7. The JSON result returns to React; React refreshes the graph and shows the updated view.

End-to-end file request

1. React sends the file name and hex bytes to POST /api/v1/models/file-classifier/predict.
2. Pydantic checks that the sample contains valid hex.
3. The saved TF-IDF vectorizer transforms it into the same feature space used in training.
4. The MLP predicts one of six file classes and a probability score.
5. The backend attaches an evidence node to the case and returns the result.

8. Hosting and running

Public demo

- Frontend: <https://rithikaraj21.github.io/crimegraph-ai/>
- Backend health: <https://crimegraph-ai-api.onrender.com/api/v1/health>
- Interactive API docs: <https://crimegraph-ai-api.onrender.com/docs>

GitHub Pages serves the built static React website. Render runs the Python FastAPI server. CORS permits the GitHub Pages origin. The frontend URL is configured at build time.

Render Free has a cold start after inactivity and an ephemeral file system. SQLite data can be lost after restart, redeploy, or spin-down. Let the API wake before the demo; refresh the site after it is online. Do not enter real or confidential data; the demo API has no sign-in.

Run the app locally (Windows PowerShell)

Terminal 1 — backend, from project root:

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r backend\requirements.txt
$env:PYTHONPATH = "backend"
python -m uvicorn app.main:app --reload --port 8000
```

Terminal 2 — frontend, from project root:

```
cd frontend
npm install
npm run dev
```

Open the Vite URL printed in the terminal (normally <http://localhost:5173>). Leave both terminals running.

Retrain and find the CSV files

Training takes longer than simply opening the hosted demo. Run from the project root with the project's Python dependencies installed:

```
$env:PYTHONPATH = "."
python models\botnet_cnn_lstm\train.py
python models\file_classifier_mlp\train.py
```

The first command trains three traffic architectures for 50 epochs each. The second trains the MLP for 50 epochs; the Random Forest has no epochs. Output CSVs:

- models\botnet_cnn_lstm\saved_weights\standalone_cnn_metrics.csv
- models\botnet_cnn_lstm\saved_weights\standalone_lstm_metrics.csv
- models\botnet_cnn_lstm\saved_weights\hybrid_cnn_lstm_metrics.csv
- models\file_classifier_mlp\saved_models\mlp_metrics.csv
- models\file_classifier_mlp\saved_models\random_forest_metrics.csv

The API uses the saved Hybrid CNN-LSTM and file MLP artifacts. Retraining locally does not automatically replace the hosted Render models; deployment of updated artifacts/code is a separate step.

9. Responsible use and limitations

- 1. **Synthetic data:** generated examples are not a substitute for properly sourced, licensed, representative data.
- 2. **Generalization:** synthetic scores do not establish field performance.
- 3. **Confidence is not certainty:** a probability can be wrong or poorly calibrated.
- 4. **Extraction errors:** patterns can miss entities or find incorrect ones; Gemini can also hallucinate.
- 5. **No authentication:** public demo API is not suitable for real cases.
- 6. **Temporary database:** Render Free can lose records after restart.
- 7. **Human review:** predictions/links are investigative leads only.
- 8. **Privacy:** never submit real personal, investigative, or confidential information.

What is implemented vs. future work

Present this honestly as a **working classroom prototype with unfinished production and validation work**. Do not describe unfinished items as features that already work.

In the current prototype	Future work—not implemented yet
React web interface and FastAPI endpoints for the sample workflows	Train and evaluate with properly sourced, representative real datasets
Two integrated classifiers and a local rule-based narrative extractor	Independent validation, calibration, and testing for generalization
Case graph backed by SQLite, with a fictional seeded demo case	Authentication and access control
Graph display, sample analysis flows, and case export	Durable hosted storage, backups, and operational monitoring

The generated benchmark scores demonstrate that the training and prediction pipeline runs; they do **not** validate accuracy on real investigations. The public demonstration is for synthetic data only. A real deployment would require the future work above, plus privacy, security, legal, and domain-expert review.

10. Viva questions and truthful answers

The five points to remember

- 1. **Purpose:** CrimeGraph organizes sample case information and shows possible links.
- 2. **Models:** It has two classifiers—one for network traffic and one for file samples.
- 3. **Data:** The model training examples are generated; the named real datasets were not used in the default runs.
- 4. **Limit:** The scores are not proof of real-world accuracy.
- 5. **Future work:** Real-data testing, independent validation, login/security, and durable hosted storage.

Answer in your own words. It is fine to give the short answer and stop; explain the technical detail only if your lecturer asks a follow-up.

Q: What does your project do? A: "It puts sample case information in one place and shows possible connections in a graph."

Q: Which datasets did you use? A: "I used computer-generated examples. I did not train on the original CTU-13, IoT-23, or Govdocs1 datasets."

Q: Why are the scores so high? A: "The generated examples have easy-to-recognize patterns. The scores show the training code works on those examples, not that it will be that accurate on real data."

Q: What models does the app use? A: "One model checks network traffic, and one model classifies file samples. The app also has a separate simple pattern finder for notes."

If asked for the names: "The traffic model is CNN-LSTM. The file model is TF-IDF plus MLP."

Q: What is an epoch? Why 50? A: "An epoch is one complete pass through the training examples. I trained the neural models for 50 epochs to meet the project requirement."

Q: What are the optimizer and loss? A: "The optimizer updates the model while it learns; the loss measures its mistakes. Both models use Adam to learn; they use a loss suited to their prediction task."

If asked for the exact names: "The traffic model uses Adam with binary cross-entropy. The file MLP uses Adam with log loss."

Q: What are trainable parameters? A: "They are the numbers the model adjusts as it learns—like learned settings."

If asked for the counts: "The traffic model has 77,953 trainable parameters; the file model has 72,774."

Q: Why use CNN-LSTM? A: "The CNN looks for nearby patterns; the LSTM looks at how patterns change across a short sequence."

Q: Why use TF-IDF before the file model? A: "The model needs numbers, not raw bytes. TF-IDF turns byte patterns into numbers the classifier can use."

Q: What do precision and recall mean? A: "Precision asks, 'When the model flags something, how often is it right?' Recall asks, 'How much of what we were looking for did it find?'"

Q: What does the graph add? A: "It lets us see how sample people, devices, places, and evidence are linked. A link is only a lead to check, not proof."

Q: Is the note extractor one of the ML models? A: "No. It uses simple text patterns by default. The app has two trained classifiers; Gemini is an optional feature."

Q: Is the project finished or production-ready? A: "It is a working classroom prototype, not a finished production system. Testing on suitable real datasets, stronger validation, login/security, and reliable hosted storage are future work."

Q: How did you make it? A: "I used AI assistance while building it, and I've been learning how the parts work. I can explain the app, its models, and what still needs improvement."

11. Five-minute demo plan

1. Open the public app and allow the free API to wake.
2. Point out the case selector, API status, and graph totals.
3. Select one graph node and explain the record/relationship panel.
4. Open "Add evidence"; show the narrative, network-flow, and hex inputs.
5. Use only synthetic defaults; submit one example if time permits.
6. Point out the graph update and model result; call it a prediction, not a fact.
7. Close with the synthetic-data and human-review limitations.

One-minute opening

"My project is called CrimeGraph AI. It organizes sample case information and shows possible links in a graph. It has one model for network traffic and one for file samples. I used generated training examples, so the scores do not tell us how accurate it would be on real cases. It is a classroom prototype, and real-data testing and security are future work."

If you forget an answer

Say: "I want to answer that accurately. My understanding is _____. I would verify the exact implementation in the training/API code before making a stronger claim." This is better than guessing.

12. Last-minute memory card

- **What it does:** organizes sample case information and shows possible links.
- **Two models:** one checks network traffic; one classifies file samples.
- **Training data:** generated examples, not the original named datasets.
- **Important limit:** the scores do not prove real-world accuracy.
- **Future work:** real-data testing, stronger validation, security, and durable storage.
- **Be honest:** say you used AI assistance and explain what you understand.